

GIT Y GITHUB

Git es un software controlador de versiones que posee una terminal con la cual vamos a poder hacer múltiples funcionalidades.

GitHub es un software en la nube que tiene múltiples funcionalidades, en la que más destaca, es permitirnos ser un repositorio para guardar software en la nube, permitiendo controlar las versiones de software tanto en mi repositorio local como en el de la nube.

Flujo de trabajo en git

Para poder llevar un control de las versiones que vamos originando, primero tenemos que ser capaces de hacer que nuestro directorio (donde se alojan los archivos que vamos a controlar) se convierta en un repositorio de git, con todas las dependencias necesarias para que git pueda controlar nuestros archivos. Para lograr eso empezamos con el comando:

git init: Al ejecutar este comando en nuestro git bash y en dirección de nuestra carpeta correspondiente, logramos que git nos genere una subcarpeta llamada “.git”. Esta subcarpeta va a contener toda la estructura necesaria para el control de versiones (historial de commits, ramas, configuración, etc). A partir de este momento, nuestra carpeta padre se convierte en un repositorio local de git y se podrá hacer lo siguiente:

1. **Rastrear cambios** en tus archivos con git add y git commit
2. **Ver el historial** de modificaciones con git log
3. **Crear ramas** para trabajar en features o experimentos con git branch
4. **Revertir cambios** a versiones anteriores si algo sale mal
5. **Pull Request**
6. **Usar .gitignore**
7. **Usar git stash**
8. **Usar git fetch**
9. **Resolución de conflictos**

1. Una vez hecho todo esto, supongamos que empezamos creando un fichero de texto simple llamado archivo.txt, pero queremos rastrear sus cambios que vamos originando. Para esto escribimos lo siguiente en la git bash:

git add archivo.txt: Este comando le dice a git que empiece a rastrear (track) el archivo y los agrega al staging. El staging es simplemente un área donde se agrupan los cambios realizados en el directorio de trabajo antes de confirmarlos (commit) al repositorio.

git commit -m “comentario”: Este comando nos permite guardar el estado actual (en este caso inicial ya que recién creamos el fichero) de nuestro archivo.txt, garantizando que no se pierda el progreso inicial. Si le hacemos un cambio a ese fichero, aquí es

donde se registra ese cambio. Este comando es esencial antes de usar el comando git push.

Una vez hecho eso supongamos que modificamos nuestro archivo.txt. Podemos usar el comando git diff para ver que texto cambio en nuestro archivo.txt. También podemos volver a aplicar los comandos git add y git commit para registrar el cambio en nuestro archivo.txt. Con git add rastrea el ultimo cambio que le hicimos y con git commit confirmamos el registro de ese cambio.

Ahora supongamos que ese fichero, que modificamos y guardamos sus cambios con ayuda de los comandos de git, queremos tenerlo en un repositorio almacenado en la nube. Aquí es donde nos ayuda GitHub. Para poder mandar nuestro archivo.txt a nuestro repositorio en GitHub desde la git bash, escribimos el siguiente comando:

git push -u origin: Este comando nos ayuda a introducir nuestro fichero (que esta commiteado) a nuestro repositorio "origin" en GitHub. El subcomando -u se usa para vincular tu rama local (la que tienes en tu directorio) con una rama remota especifica (la que está en GitHub). A ese vínculo Git lo llama tracking. Tu rama local queda "siguiendo" la rama remota **y crea la rama remota en GitHub**. Gracias a ese vínculo, después de la primera vez Git sabe tres cosas automáticamente: a donde enviar cuando haces git push, de donde traer cuando haces git pull y que tan adelantado o atrasado estas respecto a GitHub.

Entonces si deseo enviar un archivo a mi repositorio origin de GitHub, pero a una rama distinta, tendría que volver a escribir el comando completo **git push -u origin nombre-de-la-rama**. Después de eso, cada vez que estés parado en esa rama y hagas git push, Git ya sabe a cuál rama remota enviar los cambios. Veremos más adelante como ubicarse en una rama en la git bash.

Ahora imaginemos que tenemos archivos en nuestro repositorio en la nube de GitHub que queremos traerlos a nuestro repositorio local en Git. Para ello nos ayudamos de los comandos git clone y git pull:

git clone: Lo usas una sola vez cuando aún no tienes el repositorio en tu computadora.

Este comando descarga todo el repositorio desde GitHub y crea la carpeta en tu maquina con todo adentro: archivos, historial de commits, todo. Su sintaxis es:

git clone <https://github.com/MarioWindranger/mokepon.git>

git pull: Lo usas cuando ya tienes el repositorio en tu computadora y quieres traer los cambios nuevos existentes en GitHub. Su sintaxis es:

git pull

2. Ahora supongamos que deseamos ver lo que hemos modificado en nuestro archivo.txt. Para eso nos ayudaremos de lo siguientes comandos:

git log: Este comando nos sirve para ver todo el historial de cambios que hemos realizado, es decir, vemos todos los commits que hemos hecho. Estos commits van a contener su identificador único, el autor de ese commit, la fecha de cambio y una breve descripción del cambio que se hizo. Ten en cuenta que los commits aparecen ordenados del más reciente al más antiguo, es decir, el ultimo cambio que hiciste aparece primero arriba.

```
maraz2@CONSULTORIO2 MINGW64 ~/onedrive/desktop/ddd (master)
$ git log
commit bdfbe626a4acd868e4bcb84b62f7cd1990166c18 (HEAD -> master)
Author: MarioWindranger <rendonmario10@gmail.com>
Date: Thu Apr 16 15:48:31 2026 -0500

    cambio en el texto

commit df89247dcce30eaa4088f3f98beedddcaa7a56d64 (origin/master)
Author: MarioWindranger <rendonmario10@gmail.com>
Date: Wed Apr 15 16:30:17 2026 -0500

    Rename prueba to prueba.txt

commit c6a6dad67c09e8a2739ea45cbc98bdd57e379f1e
Author: MarioWindranger <rendonmario10@gmail.com>
Date: Wed Apr 15 16:27:37 2026 -0500

    Create prueba

commit f7e869f1388db0609fa7a6860a39f974cb551280
Author: MarioWindranger <rendonmario10@gmail.com>
Date: Wed Apr 15 16:07:53 2026 -0500

    commit de prueba
```

Este comando tiene a su vez subcomandos:

git log --oneline: Muestra el historial de forma resumida en una sola línea.

git log -p: Muestra el historial con los detalles de los cambios realizados en cada archivo.

git log --graph --oneline --all: Muestra el historial como un gráfico, ideal para ver ramas.

git log -- [archivo]: Muestra el historial específico de un archivo.

git blame [archivo]: Muestra qué usuario y en qué commit se modificó cada línea de un archivo.

En VSCode con la extensión “Git History”, puedes ver el historial de archivos o líneas. También se puede usar la extensión “Git Graph”.

- Usa **Git Graph** si necesitas ver la estructura del proyecto, entender la fusión de ramas o realizar operaciones de Git visualmente.
- Usa **Git History** si necesitas hacer debugging, ver quién cambió una línea de código o revisar el historial de un archivo particular.

3. En git existen dos tipos de ramas: rama principal y rama secundaria. Una rama principal es una serie de commits mientras que una rama secundaria es una serie de

commits, que se origina de un commit de la rama principal, pero que están presentes en una extensión de la rama principal. Podemos visualizarlo mejor:

Master: A → B → C → D (Rama principal)



Prueba: E → F → G (Rama secundaria)

Entonces la rama secundaria termina siendo una copia paralela de tu proyecto donde puedes trabajar libremente sin afectar la línea principal. Trabajas en tu rama secundaria (de nombre “Prueba”) sin tocar tu rama principal master. Cuando todo esté listo y funcione bien, fusionas los cambios de nuevo a master.

El uso más común es mantener master siempre estable y funcional, y experimentar o desarrollar cosas nuevas en ramas separadas. Si algo sale mal en la rama nueva, master no se ve afectado para nada.

Para manipular las ramas efectuamos los siguientes comandos:

git branch nombre-rama: Crea una nueva rama con su nombre en **TU computadora**.

git checkout nombre-rama: Moverte a esa rama.

git checkout -b nombre-rama: Crear la rama y moverte a ella al mismo tiempo.

git branch: Ver todas tus ramas.

git branch -d nombre-rama: Eliminar rama que ya no necesitas.

Si deseamos fusionar una rama con master:

git checkout master: Primero nos movemos a master

git merge nombre-rama: Se fusiona la rama secundaria con la rama principal.

Un ejemplo práctico con Mokepon

Imagina que quieres agregar una nueva criatura al juego, pero no sabes si va a quedar bien:

```
git checkout -b nueva-criatura // creas la rama y te mueves a ella
```

```
// ... trabajas, haces commits ...
```

```
git checkout master // vuelves a master
```

```
git merge nueva-criatura // fusionas cuando está listo
```

Si la criatura no quedó bien, simplemente borras la rama y master queda intacto como si nada hubiera pasado.

Las ramas son la razón por la que los equipos de desarrollo pueden trabajar varias personas al mismo tiempo en el mismo proyecto sin pisarse el trabajo entre sí. Cada quien trabaja en su propia rama y al final todo se fusiona.

4. Si algo sale mal en git y queremos volver a versiones anteriores, es decir "todo salió mal y quiero volver completamente a un punto anterior como si nada hubiera pasado", el comando es:

```
git reset --hard a3f9c12
```

Donde a3f9c12 es el código del commit al que quieres volver, que obtienes con:

```
git log --oneline
```

Ese comando borra todo lo que hiciste después de ese commit y deja tu proyecto exactamente como estaba en ese momento. Es el "botón de pánico" de Git.

La única regla importante es: si ya subiste esos commits a GitHub, no uses git reset --hard. En ese caso usa git revert porque es más seguro y no reescribe el historial remoto.

5. Supongamos que queremos fusionar dos ramas remotas que tenemos en nuestro repositorio de GitHub. GitHub te facilita esto con algo llamado Pull Request. Es la forma estándar de fusionar ramas, especialmente en equipos.

Pasos:

1. Ve a tu repositorio en GitHub.
2. Haz clic en **"Pull requests"** en el menú superior de tu repositorio.
3. Haz clic en el botón verde **"New pull request"**.
4. Veras dos selectores: base: master ← compare: prueba. Selecciona ambos.
5. GitHub te mostrará los cambios que hay entre las dos ramas. Si todo se ve bien, haz clic en **"Create pull request"**.
6. Escribe un título, luego haz clic en **"Create pull request"** nuevamente.
7. Finalmente haz clic en **"Merge pull request"** y luego **"Confirm merge"**

¡Listo! La rama prueba queda fusionada con master directamente en GitHub.

6. El .gitignore es un archivo que le dice a Git qué archivos o carpetas ignorar, es decir, que nunca los rastree ni los suba a GitHub, ya que seguramente en tu carrera como desarrollador te encontraras con muchos archivos que vas a desear ignorar para que no se git no rastree sus cambios. Hay archivos que no tienen sentido subir a GitHub como:

- Archivos con contraseñas o claves API
- La carpeta node_modules (pesa muchísimo y se puede regenerar)
- Archivos temporales del sistema como .DS_Store en Mac
- Variables de entorno como .env

Ahora veamos como se logra todo esto. Como .gitignore es un archivo entonces va a tocar crearlo. Para eso usamos el comando touch (que sirve para crear archivos vacíos usando la extensión) para crear nuestro archivo en la git bash:

```
touch .gitignore
```

Una vez creado el archivo hay que saber como se usa. Necesitamos abrir el archivo .gitignore y almacenar dentro de él, los archivos que queremos que git no rastree. Sin embargo, podemos usar un comando especial en nuestra git bash para que almacene el archivo que deseamos que git ignore sin necesidad de abrir nuestro .gitignore:

```
echo "node_modules/" > .gitignore
```

Un detalle importante de este comando:

Un detalle importante

Usa `>` y no `>>`. La diferencia es:

Comando	Qué hace
<code>echo "node_modules/" > .gitignore</code>	Escribe y reemplaza todo el contenido
<code>echo "node_modules/" >> .gitignore</code>	Agrega al final sin borrar lo que había

Como el archivo está vacío en este caso, cualquiera de los dos funciona, **pero atento con eso.**

Luego podemos verificar que nuestro archivo .gitignore almacene correctamente el archivo node_modules (el que vamos a ignorar):

```
cat .gitignore // Te debería mostrar node_modules/
```

Una vez hecho todo eso, debemos rastrear y guardar los cambios que hemos hecho en .gitignore. Eso ya lo sabemos hacer:

```
git add .gitignore
```

```
git commit -m "agregar node_modules al gitignore"
```

Y si deseamos subir todos esos cambios a nuestro repositorio remoto de GitHub:

```
git push
```

Importante: El .gitignore solo funciona con archivos que todavía no han sido commiteados. Si ya subiste un archivo a GitHub y luego lo agregas al .gitignore, Git lo sigue rastreando. Para ese caso necesitas este comando extra:

```
git rm --cached nombre-del-archivo
```

Eso le dice a Git que deje de rastrear el archivo sin borrarlo de tu computadora. Una vez hecho eso aplicas lo que vimos anteriormente.

7. Git stash es un comando que se usa en la git bash con el objetivo de generar una especie de gaveta temporal donde van a guardar cambios que aún no están listos para un commit, pero que tampoco deseas perderlos. Git guarda el stash dentro de la carpeta .git de tu proyecto, así que mientras no borres esa carpeta, tu stash estará ahí.

Imagina que estas en tu trabajo modificando un archivo cualquiera (**ej: generando un escrito**) y de repente tu jefe te dice que necesitar arreglar urgente un bug que está en otra rama. ¿Cómo harías? No puedes cambiarte de rama con cambios sin commitear porque Git no te va a guardar esos cambios en los que estabas trabajando, pero tampoco vas a hacer un commit de algo incompleto. Tu solución al apuro de tu jefe sería git stash. Entonces guardas tu archivo modificado y vas a escribir en la git bash el comando **git stash** y el escrito que agregaste desaparece de tu archivo y ese escrito se va a almacenar en la gaveta temporal que creaste con git stash. Es decir, tus archivos vuelven al estado del último commit, como si no hubieras tocado nada. Pero tus cambios están guardados en la gaveta.

Ahora supongamos que quieres verificar que hay en la gaveta guardado. Para ello, usas el siguiente comando:

```
git stash list
```

La git bash te va a mostrar algo así:

```
stash@{0}: WIP on master: a3f9c12 último commit.
```

```
stash@{1}: WIP on master: b7e2d45 commit anterior.
```

Y si deseamos recuperar los cambios guardados para que se eliminen de la gaveta temporal y vayan a escribirse nuevamente en el archivo. Aquí la gaveta queda vacía:

```
git stash pop
```

pero si deseamos recuperar los cambios guardados para que se escriban nuevamente en el archivo, pero sin eliminar esos cambios en la gaveta:

```
git stash apply
```

Si queremos eliminar lo que hay en la gaveta sin recuperar los cambios guardados para que no se escriban otra vez en nuestro archivo:

```
git stash drop
```

Advertencia: git stash solo guarda los archivos que Git ya está rastreando, es decir archivos que ya tienen al menos un commit previo.

Si tienes un archivo nuevo que nunca has commiteado, git stash lo ignora por defecto. Para incluir archivos que nunca has commiteado, pero quieres guardar en la gaveta sus cambios realizados usas el siguiente comando:

```
git stash -u
```

8. Git fetch es un comando que se usa en la git bash para obtener cambios desde GitHub, especialmente en el trabajo en equipos.

Supon que estás trabajando en un proyecto con tu equipo y tus compañeros deciden subir cambios realizados al proyecto en GitHub, pero no quieres que esos cambios afecten (se fusionen) con tu repositorio o ramas locales de Git. Tu solución es git fetch. Este comando va a descargar los cambios que hay en GitHub a tu computadora, pero no los fusiona con tu código, solo los trae y los deja esperando para que tu decidas que hacer con ellos.

Por ejemplo, en un equipo donde varias personas están subiendo cambios, puedes hacer git fetch para ver qué subieron tus compañeros sin que eso afecte tu código todavía:

```
git fetch // traes los cambios de GitHub
```

```
git diff master origin/master // comparas tu código local con lo que hay en GitHub
```

```
git merge origin/master // decides fusionar cuando estés listo
```

Recuerda que origin/master representa una copia del repositorio remoto de GitHub y es la manera adecuada de referirse al repositorio remoto de GitHub para que la git bash lo diferencie de la rama master en nuestro repositorio local de git. Ejemplo: Si tuvieras una rama remota en GitHub llamada beta y deseas compararla con tu rama local master entonces escribes en la git bash: git diff master origin/beta.

9. Por último, en tu trabajo como programador te vas a encontrar con múltiples problemas cuando trabajes en equipo. Vamos a intentar solucionar uno muy común.

Imagina que estás trabajando con un compañero y el jefe les pide que modifiquen exactamente la línea 5 de un archivo y que el resto lo dejen igual, pero que lo hagan en ramas distintas a la original. Resulta que tu usaste comandos distintos a los de tu compañero para modificar la línea 5 y al momento de que tu desees **fusionar la rama tuya y la de tu compañero** resulta que tienen dos versiones distintas del mismo archivo (la línea 5 distinta). ¿Qué crees que pasaría? Pasaría que Git no va a ser capaz de interpretar con que versión de los archivos se va a quedar ya que son diferentes en la línea 5. Entonces cuando quieras fusionar ambas ramas git te dice:

```
CONFLICT: Merge conflict in server.js
```


Automatic merge failed; fix conflicts and then commit the result.

Si te metes dentro de tu archivo, Git marca el conflicto directamente dentro del archivo así:

```
<<<<<<< HEAD
const puerto = 3000 // linea 5
=====
let puerto = 8080 // linea 5
>>>>>>> origin/master
```

Cada parte significa:

- **<<<<<<< HEAD** → tu versión local
- **=====** → separador entre las dos versiones
- **>>>>>>> origin/master** → la versión que viene de GitHub

Para resolver el problema, tienes que abrir el archivo, decidir qué versión conservar y eliminar manualmente las marcas que dejó Git. Por ejemplo, si decides quedarte con el puerto 3000:

```
const puerto = 3000
```

Borras todo lo demás incluyendo las líneas con <<<<<<<, ===== y >>>>>>>.

Luego commiteas tu resolución:

```
git add server.js
```

```
git commit -m "resolví conflicto en server.js"
```

```
git push
```

Entonces la causa de este problema es:

"Dos ramas, mismo archivo, mismas líneas, Git no sabe cuál conservar."